

Library Management System

How We Built It — A Step-by-Step Walkthrough

From the first idea to the running application —
every step, why we did it, and the concept behind it.

DBMS Capstone Project · MySQL

This document walks through the project in the exact order we built it. Each step says **What** we did, **Why** we did it, and — where a new idea appears — explains the **Concept** in plain language. There are 45 steps across 10 phases.

PHASE 1 — Understanding the Problem & Planning

1 Choose and understand the project

What: We selected the **Library Management System** from the project list and listed everything a library actually does: keep books, track copies, register members, lend and return books, charge late fines, and let members reserve books.

Why: Before touching any code, you must understand the real-world process. The database is just a mirror of real life, so we first wrote down the story in plain English.

2 Find the entities (the "nouns")

What: We underlined the important nouns in our description: *Publisher, Category, Author, Book, Copy, Member, Staff, Borrowing, Fine, Reservation*. Each becomes a table.

Why: A standard design technique — nouns usually become entities (tables) and verbs (borrow, return) become relationships or actions.

Concept — Entity: a real-world object we store data about. In a relational database each entity becomes a **table**; one row = one instance of that entity (one book, one member).

3 Decide the relationships between entities

What: We worked out how the entities connect: a Publisher has many Books; a Book has many Authors and an Author writes many Books; a Book has many physical Copies; a Member has many Borrowings; a Borrowing may have one Fine.

Why: Relationships decide where the foreign keys go and which junction tables we need.

Concept — Cardinality: how many rows on each side relate. **1:1** (one-to-one), **1:M** (one-to-many), and **M:N** (many-to-many). We deliberately included all three to cover the syllabus.

4 Choose the technology: MySQL + InnoDB

What: We chose **MySQL** as the database and the **InnoDB** storage engine for every table.

Why: InnoDB is the only common MySQL engine that supports **foreign keys** and **transactions**, both of which this project needs.

Concept — Storage engine: the component that actually stores and retrieves rows. InnoDB gives ACID transactions and referential integrity; the older MyISAM does not.

5 Decide the overall architecture (3-tier)

What: We planned three layers — a browser UI, a small server (Node/Express), and the MySQL database.

Why: A browser cannot connect to MySQL directly (no driver + security sandbox), so a server must sit in the middle.

Concept — 3-tier architecture: **Presentation** (browser) → **Application** (server/API) → **Data** (database). Each layer has one job, which keeps the system organised and secure.

PHASE 2 — Designing the Schema (on paper first)

6 Draw the ER diagram

What: We drew an **Entity–Relationship diagram** showing all 11 entities and the lines (relationships) between them, marking which side is "one" and which is "many".

Why: The ER diagram is the blueprint. It is much cheaper to fix a design on paper than after writing SQL.

Concept — ER model: a diagram of entities (boxes), attributes (their columns) and relationships (lines). "Crow's-foot" notation shows cardinality with the fork symbol meaning "many".

7 Choose primary keys (surrogate ids)

What: Every table gets an auto-incrementing id column (`book_id` , `member_id` , ...) as its primary key.

Why: A simple numeric key never changes and is easy to reference from other tables, unlike real-world values (a title or email can change).

Concept — Primary key & surrogate key: the **primary key** uniquely identifies each row and cannot be NULL (entity integrity). A **surrogate key** is an artificial id with no business meaning, used precisely because it is stable.

8 Separate "Book" (title) from "Book Copy" (physical item)

What: We split a book into two tables: `book` holds title-level facts (ISBN, title, price) once; `book_copy` holds each physical copy with its own barcode and status.

Why: A library owns several copies of the same title. Storing the title once and the copies separately avoids repeating the title, author and price for every copy.

Concept — avoiding redundancy: store each fact in exactly one place. This is the heart of good database design and of normalization.

9 Resolve the many-to-many with a junction table

What: Because a book can have many authors and an author many books, we created a `book_author` table holding pairs of (`book_id` , `author_id`).

Why: Relational tables cannot store a list of authors in one cell. A junction table turns one M:N relationship into two clean 1:M relationships.

Concept — Junction (bridge) table: the standard way to implement M:N. Its primary key is the **composite** of the two foreign keys, so each pair appears only once.

10 Model the recursive category hierarchy

What: The `category` table has a `parent_id` column that points back to `category` itself, so "Physics" can sit under "Science".

Why: Categories naturally form a tree. A self-reference models that without extra tables.

Concept — Recursive (unary) relationship: a table related to itself through a foreign key to its own primary key.

11 Model the one-to-one Fine

What: A `fine` belongs to exactly one `borrowing`. We made `fine.borrow_id` a foreign key that is also **UNIQUE**.

Why: Each loan can produce at most one fine; **UNIQUE** on the foreign key enforces the 1:1 rule.

Concept — 1:1 relationship: created by putting a **UNIQUE** constraint on a foreign key, so each parent row links to at most one child row.

PHASE 3 — Creating the Database (01_schema.sql)

12 Create the database

```
DROP DATABASE IF EXISTS library_db;  
CREATE DATABASE library_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;  
USE library_db;
```

What: Drop any old version, create a fresh database with full-Unicode character set, and make it active.

Why: `DROP IF EXISTS` lets us re-run the script anytime; `utf8mb4` supports any language/emoji in titles.

Concept — DDL: Data Definition Language (CREATE, ALTER, DROP) defines the structure of the database, as opposed to DML which handles the data inside.

13 Create the `publisher` table

```
CREATE TABLE publisher (  
  publisher_id INT AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(120) NOT NULL UNIQUE,  
  country VARCHAR(60) DEFAULT 'India'  
) ENGINE=InnoDB;
```

What: Our first, simplest table — the "one" side of Publisher→Book.

Why: Starting with tables that have no foreign keys means later tables can safely reference them.

Concept — Constraints used here: **PRIMARY KEY** (unique id), **NOT NULL** (value required), **UNIQUE** (no duplicate names), **DEFAULT** (auto value), **AUTO_INCREMENT** (auto-numbering).

14 Create `category` with a self-foreign-key

```
parent_id INT NULL,  
FOREIGN KEY(parent_id) REFERENCES category(category_id)  
    ON DELETE SET NULL ON UPDATE CASCADE
```

What: Added the recursive relationship from step 10.

Why: `ON DELETE SET NULL` means deleting a parent makes its children top-level instead of deleting them.

Concept — Referential actions: **CASCADE** (propagate the change), **SET NULL** (clear the link), **RESTRICT** (block the change) — they decide what happens to child rows when a parent changes.

15 Create `author` with a CHECK constraint

```
birth_year SMALLINT,  
CHECK (birth_year IS NULL OR birth_year BETWEEN 1000 AND 2025)
```

What: A table for authors, validating the birth year.

Why: Bad data should be rejected by the database itself, not just by the app.

Concept — Domain integrity / CHECK: guarantees a column's value stays within a valid range or set. This is "domain integrity". (Needs MySQL 8.0.16+.)

16 Create `book` with two foreign keys

```
publisher_id INT NOT NULL, category_id INT NOT NULL,  
price DECIMAL(8,2) CHECK(price >= 0),  
FOREIGN KEY(publisher_id) REFERENCES publisher(publisher_id) ON DELETE RESTRICT,  
FOREIGN KEY(category_id) REFERENCES category(category_id) ON DELETE RESTRICT
```

What: The book title table, linked to its publisher and category (the "many" side).

Why: `RESTRICT` stops anyone deleting a publisher/category that still has books, protecting integrity. `DECIMAL` stores money exactly.

Concept — Foreign key & referential integrity: a foreign key must always match an existing primary key (or be NULL). This guarantees no "orphan" book points to a missing publisher.

17 Create the `book_author` junction table

```
PRIMARY KEY (book_id, author_id),
FOREIGN KEY(book_id) REFERENCES book(book_id) ON DELETE CASCADE,
FOREIGN KEY(author_id) REFERENCES author(author_id) ON DELETE CASCADE
```

What: Implemented the M:N from step 9.

Why: The composite primary key stops duplicate pairs; `CASCADE` removes link rows automatically if a book or author is deleted.

Concept — Composite key: a primary key made of more than one column; the combination must be unique.

18 Create `book_copy` with an ENUM status

```
status ENUM('Available','Borrowed','Reserved','Lost','Damaged')
NOT NULL DEFAULT 'Available'
```

What: Each physical copy with a barcode and a status.

Why: A copy can only be in one of a few states; ENUM enforces that.

Concept — ENUM: a column type restricted to a fixed list of allowed values — a simple, readable way to enforce a small domain.

19 Create `member` (with soft-delete flag)

```
email VARCHAR(120) NOT NULL UNIQUE,
membership_type ENUM('Student','Faculty','Public'),
is_active BOOLEAN NOT NULL DEFAULT TRUE
```

What: Library members, identified uniquely by email.

Why: `is_active` lets us deactivate a member instead of deleting them, keeping their loan history.

Concept — Soft delete: marking a row inactive rather than removing it, so historical references stay valid.

20 Create **staff**

What: The librarians who issue books.

Why: So each loan can record who processed it — useful for accountability and reporting.

21 Create **borrowing** — the central transaction table

```
copy_id INT NOT NULL, member_id INT NOT NULL, issued_by INT,  
borrow_date DATE NOT NULL, due_date DATE NOT NULL, return_date DATE NULL,  
CHECK(due_date >= borrow_date)
```

What: Records every loan, joining a copy, a member and a staff member.

Why: `return_date = NULL` elegantly means "still on loan"; the CHECK stops impossible dates.

Concept — NULL: represents "unknown / not yet". Here a NULL return date is the marker for an active loan — every "current loans" query filters on it.

22 Create **fine** (1:1 with borrowing)

```
borrow_id INT NOT NULL UNIQUE,  
FOREIGN KEY(borrow_id) REFERENCES borrowing(borrow_id) ON DELETE CASCADE
```

What: A fine attached to a single loan.

Why: UNIQUE on the foreign key makes it one-to-one (step 11).

23 Create **reservation**

What: Lets a member reserve a title, with status Pending/Fulfilled/Cancelled.

Why: Completes the real-world workflow and adds another 1:M relationship from both Book and Member.

24 Add indexes

```
CREATE INDEX idx_book_title ON book(title);  
CREATE INDEX idx_copy_status ON book_copy(status);  
CREATE INDEX idx_borrow_due ON borrowing(due_date);
```

What: Added indexes on columns we search and filter often.

Why: They make those queries much faster on large data.

Concept — Index: a sorted lookup structure (like a book's index) that lets the database find rows without scanning the whole table. Trade-off: extra storage and slightly slower writes. PK/UNIQUE columns are indexed automatically.

25 Demonstrate ALTER TABLE

```
ALTER TABLE member ADD COLUMN last_updated TIMESTAMP  
DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP;
```

What: Added an audit column after the table already existed.

Why: Shows we can evolve a schema (a required skill) and auto-tracks when a member row changes.

Concept — ALTER: a DDL command that modifies an existing table's structure (add/drop/modify columns or constraints).

PHASE 4 — Checking Normalization (1NF → 2NF → 3NF)

26 Verify First Normal Form (1NF)

What: We confirmed every column holds a single atomic value — no cell stores a list.

Why: The "authors" of a book and the "copies" of a book are multi-valued, which breaks 1NF, so we moved them to the `book_author` and `book_copy` tables.

Concept — 1NF: atomic values only, no repeating groups. Multi-valued data goes into its own table.

27 Verify Second Normal Form (2NF)

What: In the composite-key table `book_author`, we checked that its only non-key column (`author_role`) depends on the whole key.

Why: If we had stored the book title here, it would depend only on `book_id` — a partial dependency. We keep the title in `book` instead.

Concept — 2NF & partial dependency: with a composite key, no non-key column may depend on just part of the key.

28 Verify Third Normal Form (3NF)

What: We checked no non-key column depends on another non-key column.

Why: Putting publisher city inside `book` would make city depend on publisher (not on the book) — a transitive dependency. We keep publisher details in `publisher`.

Concept — 3NF & transitive dependency: every non-key attribute must depend only on the primary key. Result: no redundancy and no insert/update/delete anomalies.

PHASE 5 — Inserting Sample Data (02_data.sql)

29 Insert reference data first

What: Inserted publishers, categories and authors before anything else.

Why: Foreign keys require the referenced row to exist first — you cannot add a book before its publisher exists.

Concept — Insertion order: always load "parent" (referenced) tables before "child" tables that point to them, or the foreign key check fails.

30 Insert books and their author links

What: Added 25 books, then filled `book_author` with the book–author pairs, including some books with multiple authors.

Why: To show the M:N relationship working with real data.

31 Insert copies, members and staff

What: Added 40 physical copies (some Available, some Lost/Damaged), 20 members and 5 staff.

Why: Multiple copies per title prove the book/book_copy split is useful and realistic.

32 Insert borrowings — a deliberate mix

What: Added 30 loans: some returned on time, some returned late, some still active, and some overdue.

Why: So the overdue and fine queries actually return results during the demo — empty results look like a broken system.

What: Added fines tied to the late returns and a set of reservations.

Why: Completes the dataset to ~200 records — well over the 100 minimum the brief requires.

Concept — DML: Data Manipulation Language (INSERT, UPDATE, DELETE, SELECT) works with the data inside the tables.

PHASE 6 — Creating Views (03_views.sql)

34 Create `vw_book_catalogue`

```
GROUP_CONCAT(CONCAT(a.first_name, ' ', a.last_name) SEPARATOR ', ') AS authors
```

What: A view joining book + category + publisher + authors into one tidy row, merging multiple authors into a single text field.

Why: The frontend and several queries reuse this, so we write the complex join once.

Concept — View: a stored SELECT that behaves like a virtual table. It hides complex joins behind a simple name and promotes reuse.

35 Create the availability, active-loans and fines views

What: Three more views: copies available per title, all un-retuned loans with days-overdue, and unpaid fines summed per member.

Why: These power the dashboard and tables directly, keeping the application code simple.

Concept — Derived attribute: "days overdue" isn't stored; it's computed on the fly with `DATEDIFF(CURDATE(), due_date)` — always up to date.

PHASE 7 — Writing Advanced Queries (04_queries.sql)

36 Write JOIN queries

What: Queries that combine tables — e.g. each loan with its member and book title, members who never borrowed (LEFT JOIN), and sub-categories beside parents (self join).

Why: Real questions span several tables; joins are how relational databases recombine the normalized data.

Concept — JOIN types: **INNER** (only matches), **LEFT** (all left rows + matches), **SELF** (table joined to itself). We use all three.

37 Write aggregate + GROUP BY / HAVING queries

```
GROUP BY b.book_id HAVING COUNT(br.borrow_id) >= 2
```

What: Summaries like books per category, average price, and most-borrowed books.

Why: Management reports need totals and averages, not individual rows.

Concept — WHERE vs HAVING: **WHERE** filters rows *before* grouping; **HAVING** filters *groups* after aggregation. Aggregates (COUNT/SUM) can only be tested in HAVING.

38 Write nested / correlated subqueries

```
WHERE price > (SELECT AVG(price) FROM book)      -- scalar
WHERE EXISTS (SELECT 1 FROM borrowing ... )      -- correlated
```

What: Books above the average price, members with overdue books (EXISTS), and books never borrowed (NOT IN).

Why: Some questions need a query inside a query.

Concept — Subquery types: **scalar** (one value), **correlated** (re-runs per outer row, often with EXISTS), and **IN/NOT IN** (membership test).

39 Write triggers

```
CREATE TRIGGER trg_borrow_before_insert BEFORE INSERT ON borrowing ...  
IF v_status <> 'Available' THEN SIGNAL ... -- block bad loan
```

What: Three triggers — block issuing an unavailable copy, auto-mark a copy "Borrowed" on loan, and auto-create a fine + free the copy on a late return.

Why: The rules now live in the database, so data stays correct even if accessed without our app.

Concept — Trigger: code that runs automatically on INSERT/UPDATE/DELETE. **OLD** / **NEW** hold the row before/after; **SIGNAL** raises an error that cancels the operation.

40 Write stored procedures with transactions

```
CREATE PROCEDURE sp_issue_book(...)  
DECLARE EXIT HANDLER FOR SQLEXCEPTION BEGIN ROLLBACK; RESIGNAL; END;  
START TRANSACTION; ... COMMIT;
```

What: `sp_issue_book`, `sp_return_book`, `sp_pay_member_fines`, plus a function `fn_member_balance`.

Why: They bundle multi-step operations into one safe call; the rollback handler undoes everything if any step fails.

Concept — Transaction & ACID: a group of statements that all succeed or all fail (Atomicity), leaving the data valid (Consistency), isolated from others (Isolation), and permanent once committed (Durability).

PHASE 9 — Building the Application (frontend + backend)

41 Build the backend API (Node + Express)

```
app.get('/api/books', (req,res) =>
  pool.query('SELECT * FROM vw_book_catalogue ...')
    .then(([rows]) => res.json(rows)));
```

What: A small server exposing REST endpoints that read the views and call the stored procedures.

Why: The browser talks to this server (not MySQL directly). It returns plain JSON.

Concept — API / REST endpoint: a URL the frontend calls (e.g. `/api/books`) to get or change data. JSON is the common data format exchanged.

42 Build the vanilla frontend (HTML/CSS/JS)

What: A single page with six tabs (Dashboard, Books, Members, Loans, Issue, Fines), styled with CSS, no frameworks.

Why: The brief recommends HTML/CSS/JS, and a simple UI keeps the focus on the database.

43 Connect frontend to backend with fetch()

```
const rows = await fetch('/api/books').then(r => r.json());
// then build an HTML table from the rows
```

What: JavaScript requests data from the API and renders it as tables; actions like Issue/Return POST back to the API.

Why: This is the live frontend↔database integration the evaluation rewards. Errors from triggers (e.g. "Copy not available") flow back and are shown to the user.

Concept — fetch / AJAX: JavaScript requesting data from a server in the background and updating the page without a full reload.

PHASE 10 — Running the System

44 Set up MySQL with Docker

```
docker compose up -d    # starts MySQL and auto-loads all the SQL files
```

What: A `docker-compose.yml` launches MySQL and runs the schema, data, views and procedures automatically on first start.

Why: One command gives a clean, repeatable database on any machine — no manual install.

Concept — Docker / container: a lightweight, isolated package that runs software (here MySQL) the same way everywhere, so "it works on my machine" problems disappear.

45 Start the app and open it

```
cd frontend
npm start          # starts the server on port 3000
# open http://localhost:3000 (NOT the Live Server :5500 URL)
```

What: Run the Node server and open the app in the browser; data loads live from MySQL.

Why: The page only shows data when served by the Node server, because that server is what talks to the database. Opening the raw file (or Live Server on :5500) shows blank tables — a common mistake.